Unsupervised Domain Adaptation for Image Classification

Jacob Morra
Department of Computer Science
Western University
London, ON
jmorra6@uwo.ca

Nima Hosseini Dashtbayaz
Department of Computer Science
Western University
London, ON
nhosse5@uwo.ca

Hosein Rajabi
Department of Computer Science
Western University
London, ON
hrajabi@uwo.ca

***Abstract*—In this paper, we investigate the performances of classical and more recent adversarial approaches to domain adaptation with the MNIST and MNIST-M datasets. Specifically, we formulate a pipeline, including pre-training on MNIST and model selection across four algorithms: naive domain adaptation, Unsupervised Domain Adaptation by Backpropagation (RevGrad), Adversarial Discriminative Domain Adaptation (ADDA), and Wasserstein Distance Guided Representation Learning for Domain Adaptation (WDGRL). We report that WDGRL achieves the best classification performance on MNIST-M; and that naive domain adaptation is the poorest in comparison to the domain adaptation algorithm approaches. Furthermore, we find that ADDA yields the best class separation by t-SNE on MNIST-M.**

***Index Terms*—domain shift, covariate shift, domain adaptation, image classification**

I. Introduction

Imagine that you have trained a machine learning model on a particular dataset for classification; this dataset has clean, high-resolution images of cats and dogs. You achieved 100% classification performance on this dataset. Next imagine that you attempted to employ this model in the real world by taking a picture of your pet dog, and continued to test the model on other cats and dogs; however, to your surprise, you found that the 100% accurate model only really achieves 60% accuracy on a real-world test set. Such a problem is partially described by **domain shift** [1].

Domain shift is the phenomenon that occurs when there is a change in parent distribution between a source (s) and target (t) data. Returning to our example, if the training set contained images of cats and dogs, centered, with a white background, these would, for example, belong to a different parent distribution than images taken from a smartphone, with variation in backgrounds and the centering of the animal. Here we would say that there is a *domain shift* occurring *between the training data and test data*. We refer to this specific domain shift instance as **covariate shift**. To correct this issue, one popular approach is **domain adaptation**.

Domain adaptation is a set of transfer learning techniques for solving covariate shift [2]; here "transfer learning" – see Fig. 1 – is a general approach in which a model *learns* from one domain (pre-training) and *applies that knowledge* to a target domain (fine-tuning). Specifically, domain adaptation involves a source dataset and a related target dataset – see Fig. 2. The model first *trains* and *validates* on the source dataset *with labels*, updating its internal weights according to its loss function, regularization, and optimizer; it then *tests* on a target dataset (without using the labels). Without any modifications, this is a naive approach. "Domain adaptation" techniques, which we will discuss (RevGrad, ADDA, and WDGRL), alter this via computing domain losses or by opposition (among others). In theory, domain adaptation techniques allow for greater generalization performance than the naive approach.

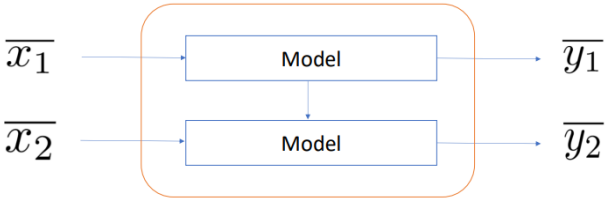


Fig. 1: Transfer learning.

Fig. 2: Domain adaptation.

Fig. 3: The MNIST and MNIST-M datasets.

In this paper, we will consider the naive approach and three domain adaptation approaches: Unsupervised Domain Adaptation by Backpropagation (RevGrad), Adversarial Discriminative Domain Adaptation (ADDA), and Wasserstein Distance Guided Representation Learning for Domain Adaptation (WDGRL). For a source dataset, we will consider the classic MNIST hand-written digits dataset [3], comprised of 60,000 hand-written digits encoded into 28×28 pixel grayscale images. For a target dataset, we will consider MNIST-M: an altered version of MNIST which combines the hand-written digit images with background patches taken from the BSDS500 dataset [4]. The task we consider is image classification; that is, given an image, predict its correct class label. Ultimately, we predict that the most recent of the three domain adaptation methods (WDGRL) will achieve the best classification performance on the test set (MNIST-M); and, that the naive approach will perform poorest of all methods.

The remainder of this report is structured as follows: First, we will contextualize our work in the literature (Background and Related Work); next, we will specify the particular objectives and methodology used (Methods); and then show all model results (Results); finally, we will summarize our work and point towards next steps (Conclusions and Future Work).

II. Background and Related Work

In this section, we will elaborate further on transfer learning, domain adaptation, and adversarial training, and briefly review the literature and trends in domain adaptation. We will also identify the gap that this paper attempts to fill.

A. Transfer learning and Domain Adaptation

Transfer learning (TL) has been an important point of interest in various machine learning tasks as it encompasses a broad range of problems and methods to solve them in a unified framework. From an abstract perspective, TL concerns the situation where the knowledge learned by a model trained on a source domain for some task is extracted and transferred to a different target domain or task [5]. To be more concise, we adopt definitions for domain and task from [5]. A domain $\mathcal{D}$ is defined as a feature space $\mathcal{X}$ and a marginal distribution $P(X)$ forming $\mathcal{D} = \{\mathcal{X}, P(X)\}$, where X is a set of instances sampled from $\mathcal{X}$. A task $\mathcal{T}$ consists of a label space $\mathcal{Y}$ and a decision function f, i.e. $\mathcal{T} = \{\mathcal{Y}, f\}$. Domain adaptation is then a branch of transfer learning methods where the source and target domains are different [5], [6]. However, there is a growing interest in *cross-domain few-shot learning* which is the case where both task and domain are different between source and target, and sometimes these methods also use the domain adaptation name [7], [8].

Similar to TL, DA has a broad range of applications in computer vision [9]–[11], NLP [12], and even scientific ML [13]. However, computer vision has been the focal point of the DA research with applications in image classification [9], [14]–[16], object detection [17], and image segmentation [11], [18]. Training robust medical models with limited data because of privacy concerns [19] and deploying models trained with synthetic data on real-world problems [11], [18] are some examples of how domain adaptation can be used in real problems. DA in computer vision can be divided into many settings. Unsupervised DA, the most common setting, assumes no labels are available in the target domain. Other settings include source-free DA [20], [21], where only the source model is available during the domain adaptation process without access to the source data, few-shot DA [22], [23], where data in the target domain is scarce, and cross-domain few-shot learning [7], [8] where source and target tasks are also different.

In this work, we are interested in unsupervised domain adaptation for image classification, and we use the term domain adaptation or DA to refer to this setting. Methods used in DA literature usually fall into one of *domain-invariant feature learning*, *domain mapping*, or *normalization statistics* methods [24]. In domain-invariant methods, the model is trained to map both source and target images to a domain-invariant feature space so that a single classifier can be applied to both source and target domains. Domain mapping methods, instead of learning domain-invariant feature representations, learn to map one domain to another, usually in the pixel level by using GANs and image-to-image translation methods. Finally, normalization methods update network normalization parameters such as batch normalization layers to better align with target domain statistics. With the introduction of adversarial DA in [9] and [10] for learning domain-invariant feature representations, domain-invariant methods have been dominant in the DA horizon. All three papers studied in this project are domain-invariant DA papers, starting with RevGrad [9] that first used the gradient reversal layers for DA, continuing with ADDA [25] that structured a framework for these methods, and ending with WDGRL [16], which proposed a novel loss function for estimating the distance between two distributions.

B. Adversarial Training

Adversarial training is a common approach in many DA methods. Inspired by adversarial games in Game Theory, it is a learning paradigm where two networks confront each other, each optimizing some min-max loss function (i.e. see Eq. 1, with noise vector z). Adversarial training is used in GANs to train a generator network (G) that produces fake images and a discriminator (D) that identifies true images from generated ones – the two networks will play an adversarial game until the generator can deceive the discriminator by generating convincing images [26].

$$\begin{aligned}\min_G \max_D L(D, G) = & E_{x \sim p_{data}} \log D(x) + \\ & E_{z \sim p(z)} \log(1 - D(G(z))),\end{aligned} \tag{1}$$

C. Research Gap

Domain adaptation has been well-explored in recent years with rich literature in many settings. As the field gets more attention, the need for comprehensive study of the available methods arises as well. Many surveys review transfer learning and domain adaptation methods from a high-level [6], [24],

[27]. However, with neural networks getting bigger and more complex methods are introduced for DA, each line of methods requires more in-depth study to understand how they behave as data complexity and network architecture changes. Adversarial methods in particular, make network optimization more complex and are very sensitive to the data and model parameters. Our interest is in studying this line of methods to see how their differences and how they behave. To the best of our knowledge, no other work studies this specific set of methods.

III. Methods

A. Research Objectives

For this work, we have the following hypothesis and corresponding objectives:

Hypothesis: Among naive, gradient-based, and oppositional approaches to domain adaptation for image classification, Wasserstein Distance Guided Representation Learning will achieve the greatest classification performance, whereas a naive approach will yield the poorest performance as it does not take advantage of domain adaptation.

O1: Find a low-dimensional mapping for the feature space of each of the algorithms considered (naive, RevGrad, ADDA, WDGRL) using t-SNE.

O2: Train, validate, and test the considered algorithms on the MNIST and MNIST-M datasets, respectively, and report the final classification performance of each.

B. Research Methodology

1) Learning pipeline: We consider the MNIST (source) and MNIST-M (target) datasets. The MNIST dataset contains 60,000 28×28 pixel images for training and 10,000 28×28 pixel images as a test set. The train set is split into two subsets, which are the train set and 20% of the train set as the validation set. All the images are divided by 255 to get mapped between [0,1]. MNIST-M is created by combining MNIST digits with the patches randomly extracted from color photos of BSDS500 as their background. It contains 59,001 training and 9,001 test images all of which are in the RGB channel. These images after preprocessing are fed into the naive model to check the results without applying any domain adaptation techniques (see next section). Thereafter, for each of the methods, we train them using the MNIST dataset and test using MNIST-M; for MNIST-M, the labels are not used by the models (unsupervised). During the training, dropout layers with 50% drop rate are used to reduce overfitting, and the optimizer is Adam. Learning rate decay is also used to get better results, and faster learning. Once trained, each model is used to predict MNIST-M labels that were not present from the beginning. Some evaluation metrics such as accuracy, confusion matrix, and t-SNE for better visualization are used to test the performance and efficiency of the proposed models.

2) Models Considered:

Naive – Here we apply transfer learning without the use of a domain adaptation algorithm. That is, a CNN trains on a source dataset, and then tests on a target dataset; in other words, no transfer learning is actually applied. We would expect that a naive approach would yield the lowest classification scores on the test set, compared to domain adaptation methods. This method serves as a baseline for determining whether domain adaptation methods are working as intended.

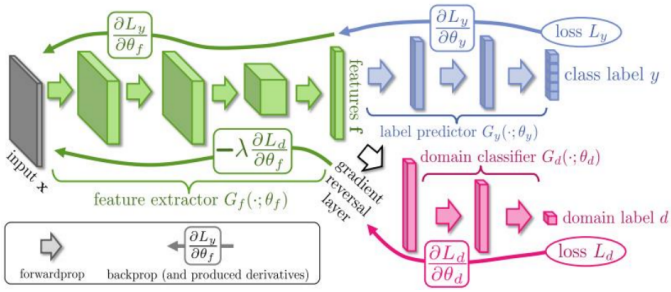


Fig. 4: RevGrad model. [9] While the discriminator network minimizes the loss L_d by updating θ_d, the gradient reversal layer reverses the gradients' direction for the feature extractor so that it maximizes L_d by updating θ_f. The classification loss L_y is still minimized by both the feature extractor and the classifier networks for the source domain.

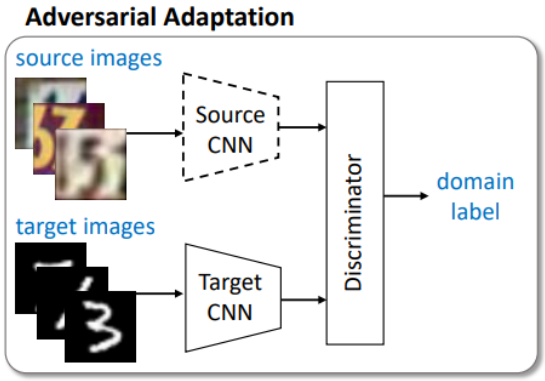


Fig. 5: ADDA uses adversarial adaptation such that a discriminator cannot reliably predict target versus source samples.

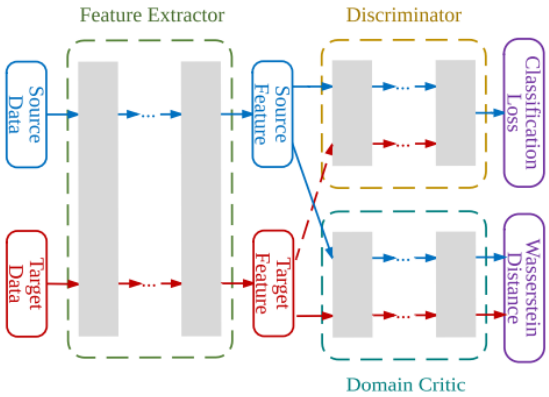


Fig. 6: Wasserstein Distance Guided Representation Learning (WDGRL)

RevGrad – In image classification CNNs, there are usually two main components: a feature extractor and a classifier. The feature extractor layers reveal more general patterns from the image, whereas classifier layers use these extracted patterns to make a class decision. But, for the low layers to be completely general, the training dataset should be diverse enough – this is almost never the case as all possible variations cannot be present in a dataset. RevGrad and other domain-invariant feature learning methods try to update the feature extractor such that it generalizes to the target domain by creating feature representations that have a minimal bias towards the patterns from either the source or the target domain. Assuming that the new representation is domain-invariant, the same classifier can be applied to representations from both domains.

RevGrad was the first work that effectively adopted adversarial training for domain adaptation. After pre-training the source model with the source domain, a *domain discriminator* network is attached to the same feature extractor. The domain discriminator is a fully connected classifier network that should tell source feature vectors from those of the target domain. By applying an adversarial loss, the feature extractor tries to deceive the discriminator by outputting representations that are not biased towards a domain, which eventually results in a domain-invariant feature space. Meanwhile, the classifier component still classifies the source images so the feature extractor should still extract meaningful patterns for classification rather than irrelevant domain-invariant representations (see Fig. 4). Specifically, for the feature extractor G_f, classifier G_y, and discriminator G_d networks with weights θ_f, θ_y, and θ_d, the error function $E(\theta_f, \theta_y, \theta_d)$ is defined as

$$\begin{aligned} E(\theta_f, \theta_y, \theta_d) = & \sum_{i=1,\ldots,n} L_y(G_y(G_f(x, \theta_f), \theta_y)) \\ & - \lambda \sum_{i=1,\ldots,n} L_d(G_d(G_f(x_i, \theta_f), \theta_d)), \end{aligned} \tag{2}$$

where L_y is the cross-entropy classification loss, L_d is the logistic discriminator loss, and λ is a hyper-parameter. The optimization objective is then

$$\begin{aligned} \hat{\theta}_f, \hat{\theta}_y &= \arg\min_{\theta_f, \theta_y} E(\theta_f, \theta_y, \hat{\theta}_d) \\ \hat{\theta}_d &= \arg\max_{\theta_d} E(\hat{\theta}_f, \hat{\theta}_y, \theta_d). \end{aligned} \tag{3}$$

This way, both the feature extractor and the classifier minimize L_d, the discriminator minimizes L_d, and the feature extractor maximizes L_d. Since optimizers minimize loss functions, an intermediate gradient reversal layer is placed between the feature extractor and the discriminator that reverses gradients $\frac{\partial L_d}{\partial \theta_f}$ by multiplying them in -1 so that G_f maximizes L_d. When the discriminator and the feature extractor converge, that is when the discriminator gets no better in identifying domains, and the feature-extractor gets no better in deceiving the discriminator, the adversarial training is done. The discriminator will no longer be needed, and the same feature extractor and classifier can be used for both domains.

ADDA: Another adversarial learning method is adversarial discriminative domain adaptation which makes use of several techniques to improve the transfer learning task. Despite the presence of domain shift, ADDA has promising results by reducing the difference between the training and test domain distributions just like the previous method. Generative Adversarial Networks (GANs) show compelling visualizations, they are not optimal on discriminative tasks and can be limited to smaller shifts. On the other hand, discriminative approaches can handle larger domain shifts, but impose tied weights on the model and do not exploit a GAN-based loss [15]. ADDA combines a discriminative model, untied weights, and a GAN loss to address the domain shift problem. First, an encoder CNN is trained to represent our source data, and also a classifier using a labeled source dataset (see Fig. 5). The classifier loss is defined by Equation 4

$$\begin{aligned} & \min_{M_s, C} \mathcal{L}_{cls}(X_s, Y_s) = \\ & \quad - \mathbb{E}_{(x_s, y_s)} \sim (X_s, Y_s) \sum_{k=1}^{k} 1_{[k=y_s]} log C(M_s(X_s)) \end{aligned} \tag{4}$$

Here M_s is the representation of the source dataset after feeding it to the encoder, and C is the classifier. An encoder should be trained to represent the target dataset on the same feature space as the source dataset. This is the same as training a GAN model involving a generator, a discriminator that the generator tries to fool the discriminator during the training to the point that the discriminator is not able to distinguish between real and fake data. However, there are some differences here such as untied weights as mentioned before. The target encoder weights are instantiated by the pre-trained source encoder weights because of the lack of target labels. The source encoder weights are fixed during adversarial adaptation, and the target encoder should be trained only. Two losses for training ADDA are defined in Eq. 5 and Eq. 6 (inverted label GAN loss).

$$\begin{aligned} & \min_{D} \mathcal{L}_{adv_D}(X_s, X_t, M_s, M_t) = \\ & \quad - \mathbb{E}_{x_s \sim x_s}[log\, D(M_s(x_s))] \\ & \quad - \mathbb{E}_{x_t \sim x_t}[log(1 - D(M_t(x_t))] \end{aligned} \tag{5}$$

$$\begin{aligned} & \min_{M_t} \mathcal{L}_{adv_M}(X_s, X_t, D) = \\ & \quad - \mathbb{E}_{x_s \sim x_t}[log\, D(M_t(x_t))] \end{aligned} \tag{6}$$

M_t is the representation of the target dataset and D is the discriminator. As the last phase of this method, the trained target encoder CNN with fixed weights and the pre-trained classifier on the source dataset could be used to predict class labels for the target dataset.

WDGRL: Wasserstein Distance Guided Representation Learning (WDGRL) relies on computing the Wasserstein distance between source and target samples [16] using a "domain critic"; this distance is minimized adversarially by way of a "discriminator" (as with other GAN-based methods such as

ADDA). The Wasserstein distance between a source and target distribution is computed as the following:

$$W_1(\mathbb{P}_h s, \mathbb{P}_h t) = \sup_{||f_w||_L \leq 1} \mathbb{E}_{\mathbb{P}_h s}[f_w(h)] - \mathbb{E}_{\mathbb{P}_h t}[f_w(h)] \quad (7)$$

Here W_1 is the Wasserstein distance, $\mathbb{P}$ and $\mathbb{E}$ represent probability and expected value, s is the source dataset, t is the target dataset, $f_w(h)$ is the wasserstein representation of the feature representation ($h = f_g(x)$), and $||f_w||_L \leq 1$ is the constraint that all representations are 1-Lipschitz [16]. In WDGRL, the domain critic and discriminator first have randomly-initialized weights. Then, for each minibatch in the training set, and for random image-label mappings between source and target data, WDGRL computes Wasserstein distances: the domain critic attempts to minimize these, and the discriminator attempts to minimize a classification loss; resulting in a "high-performance classifier for the target domain" [16].

3) Data Gathering:

The source dataset (MNIST) is taken from [28]. The target dataset (MNIST-M) is created by [29], using data from [30].

4) Data Analysis:

In addition to conducting dimensionality-reduction with t-SNE (Experiment 1) and evaluating the classification accuracy of the models (Experiment 2), we report statistics such as precision (Eq. 8), recall (Eq. 9), and f1-score (Eq. 10).

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} \quad (8)$$

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} \quad (9)$$

$$\text{F1 Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (10)$$

Precision is lowered with more false positive (FP) predictions (i.e. predicting a true label which is false); *Recall* is lowered with added false negative (FN) predictions (i.e. predicting a negative class which is positive); F1-score gives a combined precision-recall metric, which is a more complete measure than accuracy alone (Note that TP represents "True Positive").

5) Threats to Validity:

We acknowledge that, since we have trained and validated the models for a small number of epochs – and moreover, for each model we have computed only a single "run" on the test set – that the robustness of our overall results are weaker than if we were to include more train-validate-test runs; however, here we aim to empirically evaluate previous methods, and in doing so have chosen to sacrifice some of this robustness in service of faithfulness to the original papers (see Rationale).

6) Rationale:

Architecture: We use a base model CNN as per [29], with a feature extractor composed of two sets of convolutions (kernel size 5), max pooling, and ReLU activation; followed by a linear classifier which reduces to ten output classes. Convolution operations transform 3-channel RGB images into 10 feature layers, followed by 20 feature layers. Max pooling uses a filter size of 2×2. The linear classifier transforms the feature space from 320 neurons to 50, and then to 10.

Parameters: We follow the literature values for optimizers, regularization, and training epochs per each model [9], [15], [16]. In particular, we use the Adam optimizer, Dropout regularization, 30 training epochs for the naive model, and 5 training epochs for RevGrad, ADDA, and WDGRL. By default, for all PyTorch code we use a learning rate of $1E-3$ and learning rate decay of 0.1.

Models: Domain adaptation has many more models beyond the scope of this work. For our scope, we focus specifically on the class of domain adaptation models which fall into "domain-invariant feature learning". As mentioned in the background section, we have chosen this class of models because for learning domain-invariant feature representations, domain-invariant methods have been dominant in the DA horizon. For this class of models, RevGrad, ADDA, and WDGRL mark significant improvements (in this order) and are of the most popularly used methods.

IV. Results

A. System requirements, architecture, and implementation

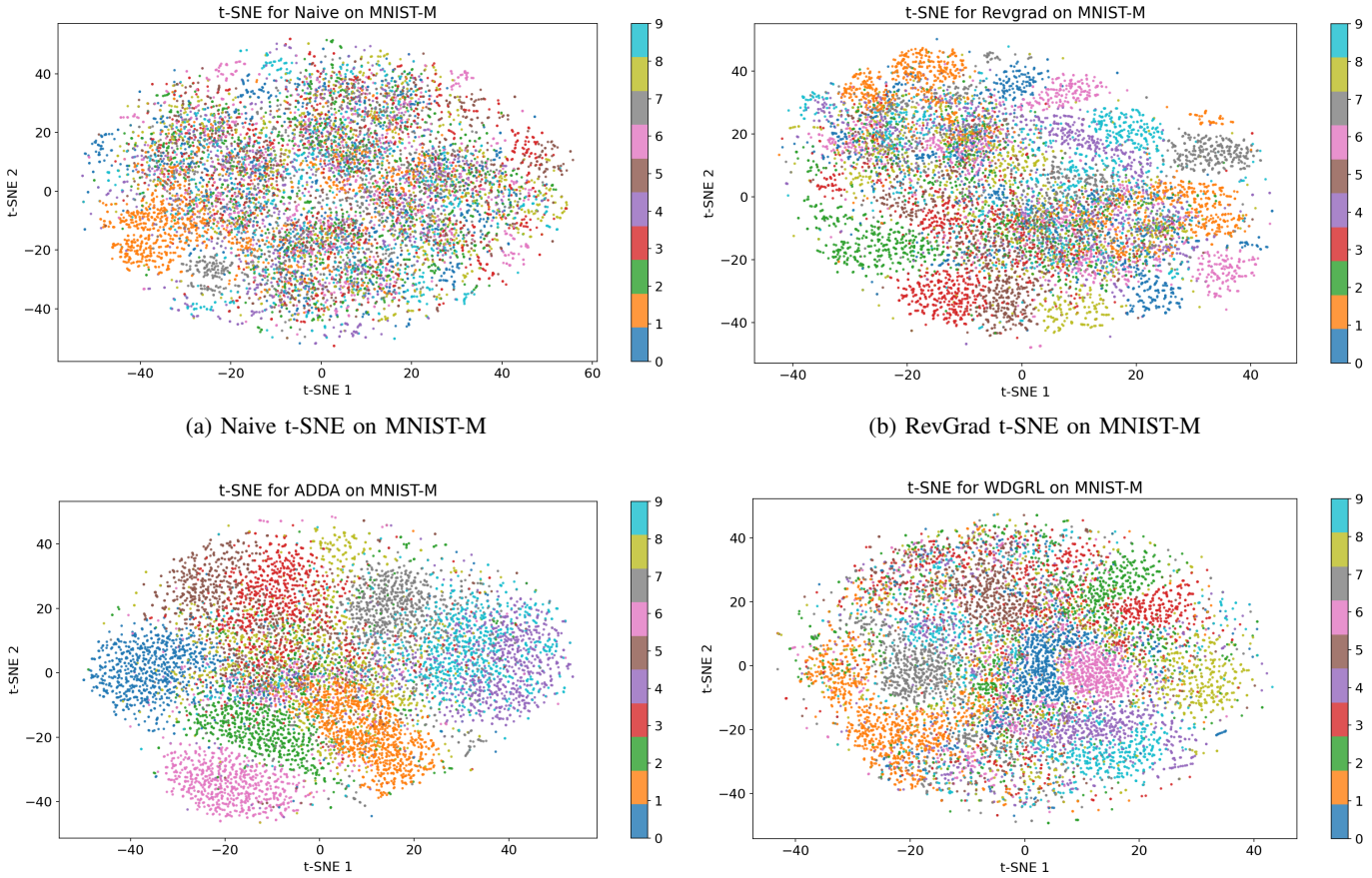
For all training and validation we used an MSI GP66 Vector laptop with 16 GB of DDR4 SDRAM and an Intel i7-12700H 14-core processor. We ran all python code using the PyCharm IDE. For code implementation and required packages, please see the attached zip file `9542_Project-Code.zip`: here we implemented the Naive method and RevGrad from scratch with TensorFlow, and added the ADDA and WDGRL methods using the PyTorch Domain Adaptation package (MIT license) [29]. All final models were trained, validated, tested, and evaluated in PyTorch to ensure consistent classification scores.

B. Experiment 1: Dimensionality reduction of methods

Please see Fig. 7. Here we have applied t-SNE as a dimensionality reduction technique for mapping features from pre-trained models into a two-dimensional space and looking for sufficient class separation. For the naive approach, we do not see any significant separation of classes in the two-dimensional space. When RevGrad is applied, importantly, we do begin to see separation between some of the classes – note that class 1, class 2, and class 3 appear to have some distinct clustering. Moving to ADDA, we find that clustering is most distinct across the 10 digit classes; and from ADDA to WDGRL, it appears that separation is less distinct. Overall, for 2d-visualization of target dataset classes, the results point towards ADDA being a superior method over all others.

C. Experiment 2: Classification of MNIST to MNIST-M

Here naive classification performance is the worst of all models, as expected. Looking at the confusion matrix (Fig. 8) we find that the trained naive model over-predicts target label 8, but otherwise is fairly accurate with other class outputs. With RevGrad, the confusion shows class over-prediction of class 4, but this is less severe than with the naive technique. ADDA tends to over-classify class 7 outputs. WDGRL, from its confusion matrix, appears to classify the most true positives. Looking at the classifier statistics (Tables I and II), we find that



(a) Naive t-SNE on MNIST-M

(b) RevGrad t-SNE on MNIST-M

(c) ADDA t-SNE on MNIST-M

(d) WDGRL t-SNE on MNIST-M

Fig. 7: Dimensionality reduction via t-SNE on MNIST-M test set for all models

WDGRL achieves the highest overall classification accuracy, and that all methods greatly outperform the naive approach – signifying that domain adaptation is working. In Table II we find that WDGRL yields the highest F1-score, and also that it outperforms all other model statistics except for the precision score of ADDA (0.73 vs 0.69), signifying that ADDA produces fewer false positive and true positive predictions combined. We also note that RevGrad achieves slightly higher F1-scores and accuracy than ADDA, which disagrees with the literature [9], [15]; though, due to our limited number of train-test runs (see "Threats to Validity") these differences may be insignificant.

TABLE I: Classification scores on MNIST to MNIST-M

Model:	Naive	RevGrad	ADDA	WDGRL
Target Accuracy	.28	.62	.61	**.68**

V. Conclusions and Future Work

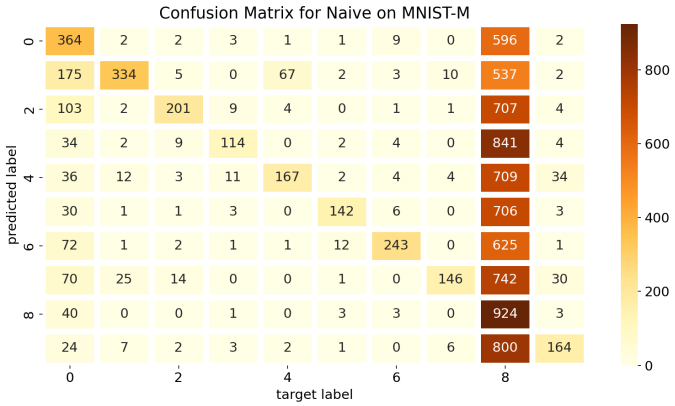
A. Conclusion

Overall, we find that WDGRL achieves superior classification performance on the transfer learning task of MNIST to MNIST-M when compared to a naive approach; and to gradient-based approaches (RevGrad) and opposition-based approaches (ADDA). We also report that, surprisingly, ADDA appears to achieve better class separation when employing two-dimensional t-SNE on the MNIST-M dataset; although we ultimately defer to the classification performance difference in claiming WDGRL as superior, we would note that, in this particular application, ADDA is a reasonable alternative if class visualization is a priority. Here we have gained some insight into the fact that some models which report a particular classification accuracy, or which claim to "out-classify" other models, may be lacking in production, or may be subject to a failure to reproduce the same performance by others.

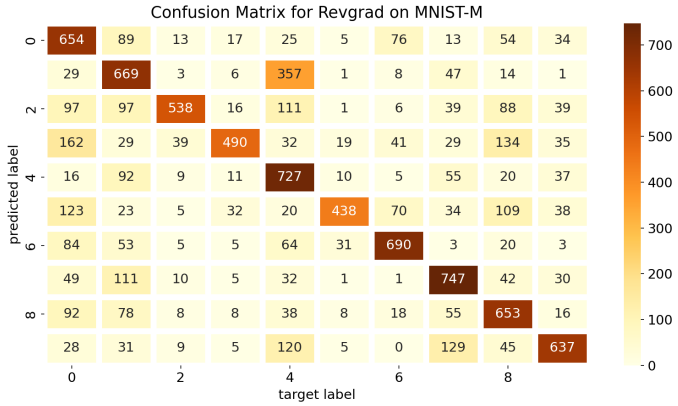
B. Future Work

One limitation of this work is that it does not delve into some recent models which report state-of-the-art classification performance on MNIST-M; these include attention-based models among others (i.e. [31]). As such, the scope of our findings is limited to the classes of adversarial and gradient-based approaches, wherein we have verified the dominance of WDGRL over ADDA, RevGrad, and the naive approach for transfer learning from MNIST to MNIST-M.

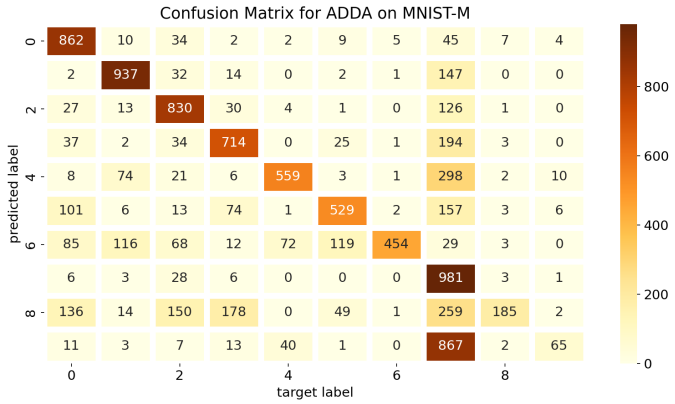
In the next iteration of this project, we will include findings from additional attention-based domain adaptation methods.



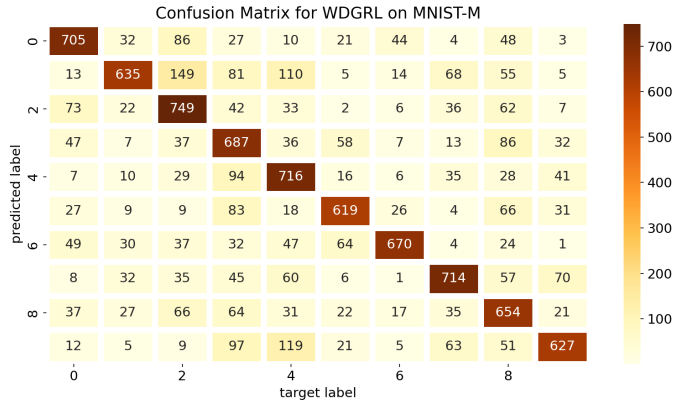
(a) Classification results on MNIST-M for Naive Approach



(b) Classification results on MNIST-M for RevGrad



(c) Classification results on MNIST-M for ADDA



(d) Classification results on MNIST-M for WDGRL

Fig. 8: Confusion matrices for test set - MNIST-M

TABLE II: Classification statistics for all models on MNIST-M

	Naive			RevGrad			ADDA			WDGRL		
CLASS	Precision	Recall	F1-score	Precision	Recall	F1-score	Precision	Recall	F1-score	Precision	Recall	F1-score
0	0.38	0.37	0.38	0.49	0.67	0.57	0.68	0.88	0.76	0.72	0.72	0.72
1	0.87	0.29	0.44	0.53	0.59	0.56	0.80	0.83	0.81	0.78	0.56	0.65
2	0.84	0.19	0.32	0.84	0.52	0.64	0.68	0.80	0.74	0.62	0.73	0.67
3	0.79	0.11	0.20	0.82	0.49	0.61	0.68	0.71	0.69	0.55	0.68	0.61
4	0.69	0.17	0.27	0.48	0.74	0.58	0.82	0.57	0.67	0.61	0.73	0.66
5	0.86	0.16	0.27	0.84	0.49	0.62	0.72	0.59	0.65	0.74	0.69	0.72
6	0.89	0.25	0.39	0.75	0.72	0.74	0.98	0.47	0.64	0.84	0.70	0.76
7	0.87	0.14	0.24	0.65	0.73	0.69	0.32	0.95	0.47	0.73	0.69	0.71
8	0.13	0.95	0.23	0.55	0.67	0.61	0.89	0.19	0.31	0.58	0.67	0.62
9	0.66	0.16	0.26	0.73	0.63	0.68	0.74	0.06	0.12	0.75	0.62	0.68
Average	0.70	0.28	0.30	0.67	0.62	0.63	**0.73**	0.61	0.59	0.69	**0.68**	**0.68**
Accuracy		.28			.62			.61			**.68**	

REFERENCES

[1] A. Farahani, S. Voghoei, K. Rasheed, and H. R. Arabnia, "A brief review of domain adaptation," in *Advances in Data Science and Information Engineering*, pp. 877–894, Cham: Springer International Publishing, 2021.

[2] H. Zhao, "On learning invariant representations for domain adaptation," *Machine Learning Blog — ML@CMU — Carnegie Mellon University*, Sept. 2019.

[3] Y. LeCun, B. Boser, J. S. Denker, D. Henderson, R. E. Howard, W. Hubbard, and L. D. Jackel, "Backpropagation applied to handwritten zip code recognition," *Neural Comput.*, vol. 1, pp. 541–551, Dec. 1989.

[4] "Papers with code - BSDS500 dataset." https://paperswithcode.com/dataset/bsds500. Accessed: 2022-12-23.

[5] F. Zhuang, Z. Qi, K. Duan, D. Xi, Y. Zhu, H. Zhu, H. Xiong, and Q. He, "A comprehensive survey on transfer learning," *Proc. IEEE*, vol. 109, no. 1, pp. 43–76, 2021.

[6] W. M. Kouw and M. Loog, "An introduction to domain adaptation and transfer learning," *arXiv preprint arXiv: Arxiv-1812.11806*, 2018.

[7] A. Zhao, M. Ding, Z. Lu, T. Xiang, Y. Niu, J. Guan, J.-R. Wen, and P. Luo, "Domain-adaptive few-shot learning," *arXiv preprint arXiv: Arxiv-2003.08626*, 2020.

[8] N. Dong and E. P. Xing, "Domain adaption in one-shot learning," in *Machine Learning and Knowledge Discovery in Databases - European Conference, ECML PKDD 2018, Dublin, Ireland, September 10-14, 2018, Proceedings, Part I* (M. Berlingerio, F. Bonchi, T. Gärtner, N. Hurley, and G. Ifrim, eds.), vol. 11051 of *Lecture Notes in Computer Science*, pp. 573–588, Springer, 2018.

[9] Y. Ganin and V. Lempitsky, "Unsupervised domain adaptation by backpropagation," Sept. 2014.

[10] Y. Ganin, E. Ustinova, H. Ajakan, P. Germain, H. Larochelle, F. Laviolette, M. Marchand, and V. S. Lempitsky, "Domain-adversarial training of neural networks," *J. Mach. Learn. Res.*, vol. 17, pp. 59:1–59:35, 2016.

[11] J. Hoffman, E. Tzeng, T. Park, J.-Y. Zhu, P. Isola, K. Saenko, A. A. Efros, and T. Darrell, "Cycada: Cycle-consistent adversarial domain adaptation," *International Conference On Machine Learning*, 2017.

[12] A. Ramponi and B. Plank, "Neural unsupervised domain adaptation in nlp-a survey," *International Conference On Computational Linguistics*, 2020.

[13] S. Goswami, K. Kontolati, M. D. Shields, and G. E. Karniadakis, "Deep transfer learning for partial differential equations under conditional shift with deeponet," *arXiv preprint arXiv: Arxiv-2204.09810*, 2022.

[14] E. Tzeng, J. Hoffman, N. Zhang, K. Saenko, and T. Darrell, "Deep domain confusion: Maximizing for domain invariance," *CoRR*, vol. abs/1412.3474, 2014.

[15] E. Tzeng, J. Hoffman, K. Saenko, and T. Darrell, "Adversarial discriminative domain adaptation," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 7167–7176, 2017.

[16] J. Shen, Y. Qu, W. Zhang, and Y. Yu, "Wasserstein distance guided representation learning for domain adaptation," July 2017.

[17] H.-K. Hsu, W.-C. Hung, H.-Y. Tseng, C.-H. Yao, Y.-H. Tsai, M. Singh, and M.-H. Yang, "Progressive domain adaptation for object detection," *Ieee Workshop/winter Conference On Applications Of Computer Vision*, 2019.

[18] Z. Murez, S. Kolouri, D. Kriegman, R. Ramamoorthi, and K. Kim, "Image to image translation for domain adaptation," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2018.

[19] S. Li, X. Sui, J. Fu, H. Fu, X. Luo, Y. Feng, X. Xu, Y. Liu, D. Ting, and R. Goh, "Few-shot domain adaptation with polymorphic transformers," *International Conference On Medical Image Computing And Computer-assisted Intervention*, 2021.

[20] S. Yang, Y. Wang, J. van de Weijer, L. Herranz, and S. Jui, "Exploiting the intrinsic neighborhood structure for source-free domain adaptation," in *Advances in Neural Information Processing Systems 34: Annual Conference on Neural Information Processing Systems 2021, NeurIPS 2021, December 6-14, 2021, virtual* (M. Ranzato, A. Beygelzimer, Y. N. Dauphin, P. Liang, and J. W. Vaughan, eds.), pp. 29393–29405, 2021.

[21] J. Huang, D. Guan, A. Xiao, and S. Lu, "Model adaptation: Historical contrastive learning for unsupervised domain adaptation without source data," in *Advances in Neural Information Processing Systems* (M. Ranzato, A. Beygelzimer, Y. Dauphin, P. Liang, and J. W. Vaughan, eds.), vol. 34, pp. 3635–3649, Curran Associates, Inc., 2021.

[22] S. Motiian, M. Piccirilli, D. A. Adjeroh, and G. Doretto, "Unified deep supervised domain adaptation and generalization," in *IEEE International Conference on Computer Vision, ICCV 2017, Venice, Italy, October 22-29, 2017*, pp. 5716–5726, IEEE Computer Society, 2017.

[23] S. Motiian, Q. Jones, S. M. Iranmanesh, and G. Doretto, "Few-shot adversarial domain adaptation," *CoRR*, vol. abs/1711.02536, 2017.

[24] W. Garrett and C. D. J., "A survey of unsupervised deep domain adaptation," *ACM Transactions on Intelligent Systems and Technology (TIST)*, 2020.

[25] E. Tzeng, J. Hoffman, K. Saenko, and T. Darrell, "Adversarial discriminative domain adaptation," Feb. 2017.

[26] A. Creswell, T. White, V. Dumoulin, K. Arulkumaran, B. Sengupta, and A. A. Bharath, "Generative adversarial networks: An overview," *IEEE signal processing magazine*, vol. 35, no. 1, pp. 53–65, 2018.

[27] L. Zhang, "Transfer adaptation learning: A decade survey," *Ieee Transactions On Neural Networks And Learning Systems*, 2019.

[28] "MNIST — torchvision main documentation." https://pytorch.org/vision/stable/generated/torchvision.datasets.MNIST.html. Accessed: 2022-12-29.

[29] Joris, "pytorch-domain-adaptation: A collection of implementations of adversarial domain adaptation algorithms."

[30] "Papers with code - BSDS500 dataset." https://paperswithcode.com/dataset/bsds500. Accessed: 2022-12-30.

[31] Y. Zuo, H. Yao, and C. Xu, "Attention-Based Multi-Source domain adaptation," *IEEE Trans. Image Process.*, vol. 30, pp. 3793–3803, Mar. 2021.